

Capítulo 8

Ecuaciones Diferenciales Ordinarias

QUIZÁS UNO DE LOS USOS MÁS COMUNES DE LOS COMPUTADORES EN FÍSICA, es su utilización en la resolución de ecuaciones diferenciales de forma numérica. En este capítulo nos enfocaremos en las técnicas básicas utilizadas para resolver ecuaciones diferenciales, de distinto tipo y para distintos problemas.

8.1. Ecuaciones de Primer Orden con una variable

Comenzaremos nuestro estudio de ecuaciones diferenciales ordinarias, enfocándonos en aquellas con **una sola** variable independiente, como por ejemplo el tiempo t , y todas las **otras variables** (por ejemplo, x , y , etc.) dependientes²⁵. El tipo más simple de ecuación diferencial ordinaria es la ecuación de primer orden con la variable dependiente t , de la forma:

$$\frac{dx}{dt} = \frac{2x}{t} + c \quad (8.1)$$

Esta ecuación puede ser resuelta exactamente de forma analítica, mediante la separación de variables. No hay necesidad de desarrollar un código para encontrar la solución. Es decir Podemos hacer simplemente hacer una suposición inicial de la forma funcional de nuestra solución, por ejemplo:

$$x(t) = k_1 t^2 - ct \quad (8.2)$$

solución que puede ser fácilmente corroborada por simple inspección. Supongamos ahora, una nueva ecuación, un poco más compleja que la anterior

$$\frac{dx}{dt} = \frac{2x}{t} + \frac{3x^2}{t^3} \quad (8.3)$$

En este caso la ecuación no puede ser separada para resolver de forma analítica, y adicionalmente es una ecuación no-lineal (potencias superiores a uno). Las ecuaciones no-lineales, muy rara vez se pueden resolver de forma analítica, sin embargo suelen ser resueltas de forma numérica. Los computadores no se preocupan si la ecuación es lineal o no, ellos resuelven en base a uno de los múltiples tipos de procedimiento establecidos para la resolución de estas ecuaciones.

La forma general de una ecuación diferencial ordinaria de primer orden esta dada por:

$$\frac{dx}{dt} = f(x(t), t) \quad (8.4)$$

en donde $f(x, t)$ es alguna función conocida. Por ejemplo de la ecuación 8.3 tenemos que $f(x, t) = \frac{2x}{t} + \frac{3x^2}{t^3}$. La variable independiente es denotada t en este ejemplo. Pero, por supuesto que esto es sólo una convención, nosotros perfectamente podríamos haber utilizado:

$$\frac{dy}{dx} = f(x, y). \quad (8.5)$$

En este capítulo mantendremos t como la variable independiente. De esta forma nosotros lo que resolveremos es la forma de $x(t)$ que corresponde a la solución de la ecuación diferencial que vamos a tratar.

²⁵Para los primeros años en Física, el tiempo es considerado la variable independiente, de la cuál todas las otras variables dependen.



Para poder dar una solución numérica completa a una ecuación diferencial, se requiere una condición inicial o condición de borde. Es decir necesitamos conocer el valor de x en un tiempo particular t , por ejemplo en $t = 0$. Así tenemos información para ese instante $x(t = 0) = a$, con a un valor conocido. Para los problemas que revisaremos debemos conocer la ecuación, y sus condiciones iniciales. A continuación se presentan los métodos más frecuentes en la resolución de ecuaciones diferenciales de primer orden.

8.2. Método de Euler

El método de Euler (Figura 8.1) es uno de los más antiguos y también uno de los más simples. Aunque hoy existen métodos mucho más precisos, Euler constituye un punto de partida **obligatorio** para comprender integradores numéricos más avanzados. Supongamos que tenemos una ecuación diferencial como la mostrada en la ecuación 8.4, y que además conocemos la condición inicial

$$x(t_0) = x_0.$$

Es decir, conocemos el valor de la función en un tiempo inicial $t = t_0$.

Antes de discretizar, conviene recordar la expansión de Taylor de una función genérica $g(\tau)$ alrededor de un punto fijo $\tau = a$:

$$g(\tau) = g(a) + (\tau - a) \left. \frac{dg}{d\tau} \right|_{\tau=a} + \frac{1}{2}(\tau - a)^2 \left. \frac{d^2g}{d\tau^2} \right|_{\tau=a} + \dots \quad (8.6)$$

En nuestro problema tomamos $g(\tau) = x(\tau)$. Para evitar confusiones de notación, distinguiremos entre la variable general τ y el instante fijo alrededor del cual expandimos la solución. Si elegimos como punto de expansión el tiempo actual t , entonces la serie queda:

$$x(\tau) = x(t) + (\tau - t) \left. \frac{dx}{d\tau} \right|_{\tau=t} + \frac{1}{2}(\tau - t)^2 \left. \frac{d^2x}{d\tau^2} \right|_{\tau=t} + \dots \quad (8.7)$$

Ahora evaluamos esta expansión en el tiempo siguiente, es decir, tomamos

$$\tau = t + h,$$

donde h es un paso temporal pequeño. Entonces obtenemos:

$$\begin{aligned} x(t+h) &= x(t) + h \left. \frac{dx}{d\tau} \right|_{\tau=t} + \frac{1}{2}h^2 \left. \frac{d^2x}{d\tau^2} \right|_{\tau=t} + \dots \\ &= x(t) + h f(x, t) + \mathcal{O}(h^2). \end{aligned} \quad (8.8)$$

En el último paso hemos usado la ecuación diferencial 8.4, es decir,

$$\frac{dx}{dt} = f(x, t),$$

y además hemos agrupado en $\mathcal{O}(h^2)$ todos los términos de orden igual o superior a h^2 . Si el paso h es suficientemente pequeño, podemos despreciar dichos términos y obtener la aproximación de Euler:

$$x(t+h) \approx x(t) + h f(x, t). \quad (8.9)$$

Esta expresión nos permite avanzar la solución desde un tiempo conocido hacia el siguiente. En particular, si conocemos el valor inicial $x(t_0) = x_0$, entonces podemos estimar el valor en $t_0 + h$ mediante

$$x(t_0 + h) \approx x(t_0) + h f(x(t_0), t_0).$$

Repitiendo este procedimiento de manera iterativa, obtenemos una sucesión de valores aproximados de la solución en tiempos equiespaciados:

$$t_n = t_0 + nh, \quad n = 0, 1, 2, \dots$$

Figura 8.1: Basilea, Suiza, 15 de abril de 1707 - San Petersburgo, Imperio ruso, 18 de septiembre de 1783. Conocido como Leonhard Euler, fue un matemático, físico y filósofo suizo. Se trata del principal matemático del siglo XVIII y uno de los más prolíficos de todos los tiempos, muy conocido por el número de Euler (e), que aparece en cálculo y física.

De este modo, el método de Euler no entrega una expresión exacta para $x(t)$ en todo instante, sino un conjunto discreto de puntos que aproximan la solución real. Mientras más pequeño sea el valor de h , mejor será, en general, la aproximación obtenida.

Si tenemos una ecuación diferencial para x y una condición inicial en $t = t_a$ y deseamos graficar $x(t)$ para valores de t entre un rango t_a y t_b , entonces podemos dividir el intervalo de t_a a t_b en un gran número de intervalos de tamaño h y usar el proceso anterior para calcular $x(t)$ –en ese intervalo–. El siguiente código ejemplifica este proceso para la ecuación 8.3.

```

1  from math import sin
2  import matplotlib.pyplot as plt
3  from numpy import arange
4
5  def f(x, t): return -x**3 + sin(t)
6
7  a, b = 0.0, 10.0  #Inicio y Final del intervalo
8  N = 1000          #Numero de pasos
9  h = (b-a)/float(N) #Tamano del paso
10 x = 0.0           #Condicion inicial
11
12 tpoints = arange(a, b, h) #Puntos equiespaciados en el tiempo
13 xpoints = []              #Lista para almacenar la soluci'on
14
15 for t in tpoints:         #Resolvemos con Euler
16     xpoints.append(x)
17     x += h*f(x, t)
18
19 #Graficamos el resultado
20 plt.plot(tpoints, xpoints)
21 plt.xlabel("$t$")
22 plt.ylabel("$x(t)$")
23 plt.show()

```

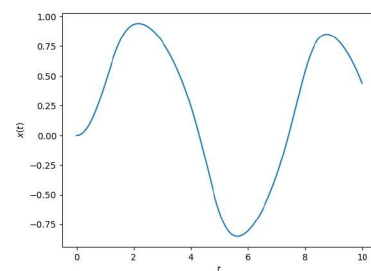


Figura 8.2: Resultado del método de Euler para resolver la ecuación diferencial 8.3

Los resultados para este código son presentados en la Figura 8.2. Si bien los resultados no son malos en el método de Euler, uno prácticamente no lo utiliza, ya que con muy poco esfuerzo adicional se consigue un método mucho más preciso y eficaz, el llamado método de Runge-Kutta.

8.3. El método de Runge-Kutta

La primera idea que se piensa para mejorar el método de Euler, es incorporar términos de orden superior en la serie de Taylor. Es decir obtendríamos términos que tienen la siguiente forma en nuestra expansión:

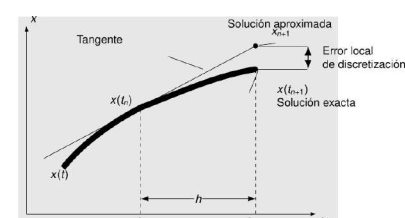
$$\frac{1}{2}h^2 \frac{d^2x}{dt^2} = \frac{1}{2}h^2 \frac{df}{dt}. \quad (8.10)$$

Si bien, la ecuación anterior nos brindaría una solución más precisa para $x(t+h)$, **no es de mucha utilidad** ya que requiere que conozcamos la derivada df/dt , la que se puede calcular si tenemos sólo la forma explícita de $x(t)$ y que en este caso, sería conocer la solución de antemano. O de forma coloquial: *"nos estaríamos pisando la cola, al tratar de avanzar"*, ya que se requiere conocer $x(t)$, que es justamente nuestra búsqueda original.

Para sobrepasar este impedimento, Carl Runge (Figura 8.3) y Martin Kutta (Figura 8.5) desarrollaron un método –conocido hoy en día como el método de Runge-Kutta (RK)–, alrededor del año 1900. RK es realmente un *set* de métodos, de echo el método de Euler es equivalente al método RK de primer orden (RK1). El método de Euler –o RK1– es fácilmente representable gráficamente, como se muestra en la figura 8.4. La curva representa el valor real de $x(t)$, que es la función que estamos interesados en calcular. La ecuación diferencial $dx/dt = f(x, t)$ nos dice que la pendiente es igual a la función $f(x, t)$ así que dado el valor de x en un tiempo t podemos calcular la pendiente en ese punto. Luego extrapolamos la pendiente a un tiempo $t+h$ y eso nos dará un valor estimado de $x(t+h)$. Si la curva $x(t)$ fuera una recta, este método sería perfecto, pero en el caso curvo, la estimación es solo una aproximación, lo que conlleva a un error en nuestros resultados.



Figura 8.3: Carl Runge (30 de agosto de 1856 - 3 de enero de 1927) fue un matemático, físico y espectroscopista alemán. Fue codesarrollador y co-epónimo del método de Runge-Kutta en el campo conocido actualmente como análisis numérico.



Supongamos ahora que hacemos el mismo cálculo pero usando la pendiente en el punto medio $t + h/2$ para la extrapolación. Con esto en consideración, nuestra estimación de $x(t+h)$ sería mucho mejor. Esta es la base del método RK de segundo orden (RK2). Desde un punto de vista matemático, el método consiste en realizar una expansión de Taylor en torno a $t + h/2$ para obtener el valor $x(t+h)$.

$$x(t+h) = x(t+h/2) + \frac{h}{2} \left(\frac{dx}{dt} \right)_{t+h/2} + \frac{1}{8} h^2 \left(\frac{d^2x}{dt^2} \right)_{t+h/2} + \mathcal{O}(h^3) \quad (8.11)$$

De forma similar podemos derivar una expresión para $x(t)$, expandiendo en torno a $t + h/2$:

$$x(t) = x(t+h/2) - \frac{1}{2} h \left(\frac{dx}{dt} \right)_{t+h/2} + \frac{1}{8} h^2 \left(\frac{d^2x}{dt^2} \right)_{t+h/2} + \mathcal{O}(h^3) \quad (8.12)$$

Es importante notar que las expansiones se hacen en torno al mismo punto, los que no permite realizar el álgebra entre nuestros resultados. Ahora, restando la ecuación 8.12 con 8.11, y reagrupando los términos obtenemos:

$$\begin{aligned} x(t+h) &= x(t) + h \left(\frac{dx}{dt} \right)_{t+h/2} + \mathcal{O}(h^3) \\ &= x(t) + hf(x(t+h/2), t+h/2) + \mathcal{O}(h^3). \end{aligned} \quad (8.13)$$

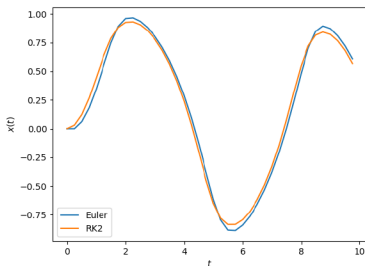
El término en h^2 ya no está y el error asociado ahora es $\mathcal{O}(h^3)$, así nuestra aproximación es más precisa que antes. Si bien esto luce prometedor, aún necesitamos conocer un término que no tenemos disponible $x(t+h/2)$, para soslayar este inconveniente podemos utilizar el método de Euler mediante $x(t+h/2) = x(t) + \frac{1}{2} hf(x, t)$ para luego sustituirlo en la ecuación previa. Así, nuestro set de ecuaciones esta dado por:

$$k_1 = hf(x, t) \quad (8.14)$$

$$k_2 = hf\left(x + \frac{1}{2}k_1, t + \frac{1}{2}h\right) \quad (8.15)$$

$$x(t+h) = x(t) + k_2 \quad (8.16)$$

En donde debemos evaluar primero k_1 , y k_2 para finalmente encontrar el valor de $x(t+h)$. Estas son las ecuaciones de RK de segundo orden (RK2). Este método es levemente más complicado que el método de Euler en términos de programación. El código a continuación muestra los resultados para ambos métodos para un valor bajo de en el número de pasos N .



```

1  from math import sin
2  import matplotlib.pyplot as plt
3  from numpy import arange
4
5  def f(x, t): return -x**3 + sin(t)
6
7  a, b = 0.0, 10.0    #Inicio y Final del intervalo
8  N = 40              #Numero de pasos
9  h = (b-a)/float(N)  #Tamano del paso
10 x = 0.0              #Condicion inicial
11
12 tpoints = arange(a, b, h) #Puntos equiespaciados en el tiempo
13 euler_xpoints = []        #Lista para almacenar la soluci'on
14 rk2_xpoints = []          #Lista para almacenar la soluci'on
15
16 #Euler
17 for t in tpoints:
18     euler_xpoints.append(x)
19     x += h*f(x, t)
20
21 x = 0.0

```

Figura 8.6: Resultado de la comparación de distintos métodos; Euler, RK2, y RK4 para resolver la ecuación diferencial 8.3.

```

22
23 #RK2
24 for t in tpoints:
25     rk2_xpoints.append(x)
26     k1 = h*f(x, t)
27     k2 = h*f(x+0.5*k1, t+0.5*h)
28     x += k2
29
30 #Graficamos ambos resultados
31 plt.plot(tpoints, euler_xpoints, label="Euler")
32 plt.plot(tpoints, rk2_xpoints, label="RK2")
33 plt.xlabel("$t$")
34 plt.ylabel("$x(t)$")
35 plt.legend()
36 plt.show()
37
38
39

```

La figura 8.6 muestra una comparación del método de Euler con RK2, se puede observar muy poca diferencia, en donde el error asociado a RK2 es mucho más bajo, por lo que numéricamente mejor. Si bien es muy poco lo que debe modificarse en términos de programación, es interesante ir un paso más allá y mejorar la aproximación de RK2, con una mayor cantidad de datos.

Ejercicio:

Pruebe el código anterior modificando la ecuación diferencial por alguna de las siguientes y compare los resultados obtenidos con ambos métodos.

- $\frac{dx}{dt} = \sin^2(x) + 2x$
- $\frac{dx}{dt} = \exp(x^2) + 2 \cos(t)$

8.3.1. El método de Runge-Kutta de 4to orden

Volviendo a tomar la expansión de Taylor en torno a varios puntos, y haciendo combinación lineal de ellos, podemos ordenar los términos h^3 , h^4 , etc. De esta forma podremos entonces cancelar ciertos términos de las ecuaciones y tener un método más preciso para resolver las ecuaciones diferenciales. En general un balance adecuado en rendimiento y precisión para el método de Runge-Kutta es el 4to orden (RK4). Las ecuaciones en este método son:

Note que la precisión acá, esta dictada por los términos de nuestra expansión de Taylor que vamos añadiendo.

$$k_1 = hf(x, t) \quad (8.17)$$

$$k_2 = hf\left(x + \frac{1}{2}k_1, t + \frac{1}{2}h\right) \quad (8.18)$$

$$k_3 = hf\left(x + \frac{1}{2}k_2, t + \frac{1}{2}h\right) \quad (8.19)$$

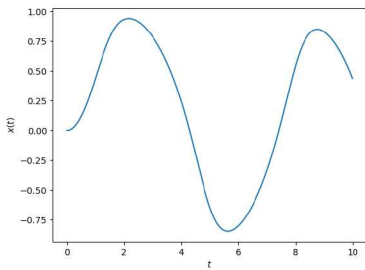
$$k_4 = hf(x + k_3, t + h) \quad (8.20)$$

$$x(t + h) = x(t) + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (8.21)$$

Ejercicio:

Desarrolle el método RK4 a partir de las expansiones previamente descritas, y compruebe –lápiz y papel– que las ecuaciones en 8.21 son correctas.

El método RK4 es uno de los más comunes para resolver ecuaciones diferenciales ordinarias. Es preciso en términos de h^4 y lleva un error $\mathcal{O}(h^5)$. Si bien la derivación puede ser

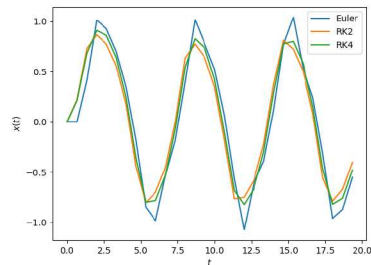


tediosa de realizar—Ejercicio—, las ecuaciones finales son relativamente simples. En general este es el primer método que se aprende a utilizar cuando se requiere resolver una ecuación diferencial ordinaria en el computador. Es simple de programar, y da buenos resultados. Es un *caballito de batalla* para resolver ecuaciones diferenciales ordinarias, y uno de los mejores algoritmos conocidos de su tipo.

```

1  from math import sin
2  import matplotlib.pyplot as plt
3  from numpy import arange
4
5  def f(x, t): return -x**3 + sin(t)
6
7  a, b = 0.0, 10.0    #Inicio y Final del intervalo
8  N = 1000           #Numero de pasos
9  h = (b-a)/float(N) #Tamano del paso
10 x = 0.0             #Condicion inicial
11
12 tpoints = arange(a, b, h) #Puntos equiespaciados en el tiempo
13 xpoints = []             #Lista para almacenar la soluci\on
14
15 #RK4
16 for t in tpoints:
17     xpoints.append(x)
18     k1 = h*f(x, t)
19     k2 = h*f(x+0.5*k1, t+0.5*h)
20     k3 = h*f(x+0.5*k2, t+0.5*h)
21     k4 = h*f(x+k3, t+h)
22     x += (k1+2*k2+2*k3+k4)/6.0
23
24 #Graficamos ambos resultados
25 plt.plot(tpoints, xpoints)
26 plt.xlabel("$t$")
27 plt.ylabel("$x(t)$")
28 plt.show()

```



A partir de la implementación presentada en el código, el resultado para el método RK4 es presentado en el gráfico 8.7. Uno podría comparar todos los métodos para distintos valores de N , y distintos intervalos para ver como es la precisión en cada uno de estos métodos en general. Una pequeña modificación al programa, cambiar el rango y el número de pasos por ejemplo, nos permite observar los distintos comportamientos para cada uno de los métodos implementados.

Consideremos por ejemplo un rango más amplio, y un valor de N más bajo para la solución de nuestra ecuación diferencial 8.3.

```

1  from math import sin
2  import matplotlib.pyplot as plt
3  from numpy import arange
4
5  def f(x, t): return -x**3 + sin(t)
6
7  a, b = 0.0, 20.0    #Inicio y Final del intervalo
8  N = 30             #Numero de pasos
9  h = (b-a)/float(N) #Tamano del paso
10 x = 0.0             #Condicion inicial
11
12 tpoints = arange(a, b, h) #Puntos equiespaciados en el tiempo
13 euler_xpoints = []        #Lista para almacenar la soluci\on
14 rk2_xpoints = []          #Lista para almacenar la soluci\on
15 rk4_xpoints = []          #Lista para almacenar la soluci\on
16
17 #Euler
18 for t in tpoints:
19     euler_xpoints.append(x)
20     x += h*f(x, t)
21
22 x = 0.0              #Restaura condicion inicial para RK2
23

```

Figura 8.8: Resultado del método de Euler para resolver la ecuación diferencial 8.3

```

24 #RK2
25 for t in tpoints:
26     rk2_xpoints.append(x)
27     k1 = h*f(x, t)
28     k2 = h*f(x+0.5*k1, t+0.5*h)
29     x += k2
30
31 x = 0.0 #Restaura condicion inicial para RK4
32
33 #RK4
34 for t in tpoints:
35     rk4_xpoints.append(x)
36     k1 = h*f(x, t)
37     k2 = h*f(x+0.5*k1, t+0.5*h)
38     k3 = h*f(x+0.5*k2, t+0.5*h)
39     k4 = h*f(x+k3, t+h)
40     x += (k1+2*k2+2*k3+k4)/6.0
41
42 #Graficamos ambos resultados
43 plt.plot(tpoints, euler_xpoints, label="Euler")
44 plt.plot(tpoints, rk2_xpoints, label="RK2")
45 plt.plot(tpoints, rk4_xpoints, label="RK4")
46 plt.xlabel("$t$")
47 plt.ylabel("$x(t)$")
48 plt.legend()
49 plt.show()

```

Se puede observar en la figura 8.8 las diferencias entre los distintos códigos, por construcción es claro que RK4 es el más preciso²⁶, y pese a no observarse inicialmente una diferencia sustancial, la diferencia con el método de Euler se hace más evidente a medida que el tiempo avanza.

Ejercicio:

Haga cambios en el código, pruebe distintos parámetros para el rango, y observe lo que ocurre, podrá notar mayores diferencias e incluso fallas en el resultado para variaciones no tan pequeñas del número de intervalos y el rango seleccionado.

²⁶Uno de los detalles importantes de los métodos RK, es que estos deben estar muy bien implementados, ya que pequeños errores en la implementación no son fáciles de detectar en los resultados.

8.4. Soluciones en rangos infinitos

Hasta ahora, hemos tratado rangos bien definidos en el tiempo t , por ejemplo (t_a, t_b) . Sin embargo existen muchas aplicaciones y problemas en donde estos rangos suelen ser importantes para casos en que $t \rightarrow \infty$. En estos casos no es directo el uso de los métodos definidos anteriormente, dado que necesitaríamos un número infinito de pasos para alcanzar $t \rightarrow \infty$. A pesar de lo anterior, existe un **pequeño truco** para poder conseguir este resultado. Para ello definamos:

$$u = \frac{t}{1+t} \quad (8.22)$$

O de forma equivalente:

$$t = \frac{u}{1-u} \quad (8.23)$$

Así, cuando $t \rightarrow \infty$ el valor de $u \rightarrow 1$. De esta forma, y por medio del uso de la regla de la cadena, podemos escribir la ecuación diferencial original $dx/dt = f(x, t)$, de la siguiente forma:

$$\frac{dx}{du} \frac{du}{dt} = f(x, t) \rightarrow \frac{dx}{du} = \frac{dt}{du} f\left(x, \frac{u}{1-u}\right). \quad (8.24)$$

Tenemos que la derivada $dt/du = \frac{1}{(1-u)^2}$. Entonces, al reemplazar dt/du en la ecuación previa nos queda:

$$\frac{dx}{du} = g(x, u) = \frac{1}{(1-u)^2} f\left(x, \frac{u}{1-u}\right). \quad (8.25)$$

A partir de este resultado, podemos construir una nueva función $g(x, u)$ que podemos utilizar para resolver $x(u)$ mediante los métodos descritos previamente. Así, mediante $x(u)$, podemos convertir nuestro resultado en una solución para t , i.e., $x(u) \rightarrow x(t)$.

Ejercicio:

Escriba un código para RK4 utilizando una solución de la ecuación diferencial

$$\frac{dx}{dt} = \exp(-x) \cdot \sin(\omega \cdot t)$$

en un rango de $t \in (0, \infty)$, para ello pruebe distintos valores de ω en su ecuación.

8.5. Ecuaciones con más de una variable

Hasta ahora hemos considerado EDO de una sola variable, que en general hemos llamado x , y que depende de la variable temporal t . Sin embargo en muchos problemas, veremos que necesitamos resolver *de forma simultánea* ecuaciones diferenciales en donde la derivada depende de una o más de los términos involucrados. Consideremos por ejemplo:

$$\frac{dx}{dt} = xy - x, \quad \frac{dy}{dt} = y - xy + \sin^2(\omega t) \quad (8.26)$$

Es importante notar que sigue existiendo una variable independiente, en este caso t para nuestro sistema de ecuaciones a modelar. Note también que continúan siendo ecuaciones diferenciales ordinarias, **no** parciales. De forma general podemos escribir este sistema como:

$$\frac{dx}{dt} = f_x(x, y, t), \quad \frac{dy}{dt} = f_y(x, y, t) \quad (8.27)$$

acá f_x y f_y son, posiblemente, funciones no lineales dependientes de x , y , y t . O para mayores dimensiones podemos generalizar esta función como ²⁷:

$$\frac{d\mathbf{r}}{dt} = \mathbf{f}(\mathbf{r}, t) \quad (8.28)$$

con $\mathbf{r} = (x, y, z, \dots)$ y $\mathbf{f}$ es una función vectorial de la forma:

$$\mathbf{f}(\mathbf{r}, t) = (f_x(\mathbf{r}, t), f_y(\mathbf{r}, t), f_z(\mathbf{r}, t), \dots). \quad (8.29)$$

Este tipo de ecuaciones, no tienen una dificultad mucho mayor, computacionalmente, si las comparamos con el caso de una sola variable. En general la expansión de Taylor de nuestro vector $\mathbf{r}$ es de la forma:

$$\mathbf{r}(t+h) = \mathbf{r}(t) + h \frac{d\mathbf{r}}{dt} + \mathcal{O}(h^2) = \mathbf{r}(t) + h\mathbf{f}(\mathbf{r}, t) + \mathcal{O}(h^2) \quad (8.30)$$

Despreciando términos para h^2 , podemos obtener el método de Euler para sistemas de múltiples variables, $\mathbf{r}(t+h) = \mathbf{r}(t) + h\mathbf{f}(\mathbf{r}, t)$. Así si procedemos de igual forma para los otros casos podemos generalizar cada método, en particular RK4 queda de la forma:

$$\mathbf{k}_1 = h\mathbf{f}(\mathbf{r}, t) \quad (8.31)$$

$$\mathbf{k}_2 = h\mathbf{f}\left(\mathbf{r} + \frac{1}{2}\mathbf{k}_1, t + \frac{1}{2}h\right) \quad (8.32)$$

$$\mathbf{k}_3 = h\mathbf{f}\left(\mathbf{r} + \frac{1}{2}\mathbf{k}_2, t + \frac{1}{2}h\right) \quad (8.33)$$

$$\mathbf{k}_4 = h\mathbf{f}(\mathbf{r} + \mathbf{k}_3, t + h) \quad (8.34)$$

$$\mathbf{r}(t+h) = \mathbf{r}(t) + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (8.35)$$

Una de las grandes ventajas de Python es la simplicidad en el manejo de sus arreglos, como se vio en el capítulo 4. Esto nos facilita toda la aritmética con los vectores simplificando la comprensión e implementación del método RK4.

²⁷Note que utilizaremos formato negrita $\mathbf{r}$ para referirnos a un vector.

8.6. Ecuaciones Diferenciales Ordinarias de segundo orden

Si bien hasta ahora hemos examinado ecuaciones de primer orden, en física las ecuaciones más observadas corresponden a las ecuaciones de segundo orden. Existe una manera muy simple de tomar esta ecuación de segundo orden y dividirla en ecuaciones de primer orden. Lo que nos permite entonces resolver las ecuaciones de segundo orden como si fuesen un set de ecuaciones diferenciales de primer orden.

Consideremos el caso mas simple, donde tenemos una sola variable dependiente x . La forma general para la ecuación diferencial de segundo orden esta dada por:

$$\frac{d^2x}{dt^2} = f\left(x, \frac{dx}{dt}, t\right) \quad (8.36)$$

Esto indica que la segunda derivada puede ser una función arbitraria cualquiera, que depende de x , t y la derivada dx/dt . Por ejemplo podríamos tener algo como:

$$\frac{d^2x}{dt^2} = \frac{1}{x} \left(\frac{dx}{dt}\right)^2 + 2\frac{dx}{dt} - x^3 \exp(-4t) \quad (8.37)$$

Ahora, si definimos una nueva cantidad y dada por $dx/dt = y$ (y este es el truco), la ecuación 8.36 se puede reescribir como:

$$\frac{dy}{dt} = f(x, y, t) \quad (8.38)$$

De esta forma, entonces, las ecuaciones $dx/dt = y$ y $dy/dt = f(x, y, t)$ son equivalente a la ecuación de segundo orden 8.36. De esta forma hemos transformado nuestra ecuación de segundo orden en dos ecuaciones simultaneas de primer orden. Y ya sabemos como resolver este tipo de ecuaciones.

De igual forma podemos generalizar el procedimiento anterior para ordenes superiores, por ejemplo una función de la forma:

$$\frac{d^2x}{dt^3} = f\left(x, \frac{dx}{dt}, \frac{d^2x}{dt^2}, t\right) \quad (8.39)$$

y luego, definiendo $dx/dt = y$, $dy/dt = z$, y transformando a $dz/dt = f(x, y, z, t)$, uno consigue desacoplar la ecuación de tercer orden en tres ecuaciones de primer orden simultáneas. Por último, para ecuaciones de orden superior y con multiples variables, tambien es posible resolver utilizando las ecuaciones escritas en forma vectorial, donde:

$$\frac{d^2\mathbf{r}}{dt^2} = \mathbf{f}\left(\mathbf{r}, \frac{d\mathbf{r}}{dt}, t\right), \quad (8.40)$$

que es equivalente a las ecuaciones de primer orden

$$\frac{d\mathbf{r}}{dt} = \mathbf{s}, \quad \frac{d\mathbf{s}}{dt} = \mathbf{f}(\mathbf{r}, \mathbf{s}, t) \quad (8.41)$$

Y así de forma general, un sistema inicial de n ecuaciones de m -ésimo orden, se transforma en un sistema de $m \times n$ ecuaciones simultáneas de primer orden, las que pueden ser resueltas con un método standard.

8.7. Paso de tiempo variable

Hasta ahora los métodos revisados tienen un paso de tiempo constante a medida que hacemos la evolución de la solución de nuestra ecuación diferencial. Existen situaciones, en las que estos pasos de tiempo son difíciles de tratar cuando son fijos, ya que valores muy pequeños retardan mucho el cálculo y valores muy grandes se alejan de la precisión esperada, esto es de gran complejidad si el problema presenta dos etapas en donde uno requiere mayor o menor precisión a medida que vamos resolviendo la ecuación.

Para resolver este tipo de problemas, podemos usar el método llamado *adaptive step sizes* (tamaño de paso adaptativo). La idea básica de este método es variar el paso h de forma que el error introducido por unidad de intervalo en t sea prácticamente constante. Así donde la solución requiere mayor precisión uno puede reducir el tamaño de h –para hacerlo debemos aumentar el número de pasos–, y donde no se requiera aumentar el tamaño.

Consideremos un valor inicial de h , y realizamos –usando, por ejemplo RK4– dos pasos en nuestra solución, es decir:

$$f(t) \xrightarrow{\delta t} f(t + \delta t) \xrightarrow{\delta t} f(t + 2\delta t)$$

si consideramos ahora volver a nuestro punto de partida, y en lugar de avanzar δt en dos ocasiones, avanzamos $2\delta t$ en una sola ocasión, entonces tendríamos una nueva estimación:

$$f(t) \xrightarrow{2\delta t} f(t + 2\delta t)$$

Así tenemos la estimación de $f(t + 2\delta t)$ calculada de dos formas distintas. La comparación de estos valores, será clave para decirnos que tan preciso es el cálculo que estamos realizando. Para cada uno de estos casos tendremos errores asociados provenientes del error del método utilizado, en este caso RK4.

Para el caso de dos pasos de tamaño δt , tenemos que $f(t + 2\delta t) = f_1 + 2c(\delta t)^5$, y para el otro caso tendremos que $f(t + 2\delta t) = f_2 + 32c(\delta t)^5$. De esta forma tenemos que $f_1 = f_2 + 30c(\delta t)^5$. Esto implica que $\epsilon = c(\delta t)^5 = \frac{1}{30}(f_1 - f_2)$ es el error por paso en pasos de tamaño δt . Ahora, debemos definir el tamaño de δt para que nuestros pasos tenga el error objetivo que nosotros deseamos. Sea δu este tamaño ideal, así

$$\zeta = c(\delta u)^5 = c(\delta t)^5 \left(\frac{\delta u}{\delta t} \right)^5 = \frac{1}{30}(f_1 - f_2) \left(\frac{\delta u}{\delta t} \right)^5$$

Ahora supongamos que la precisión objetivo por unidad de tiempo para nuestros cálculos es η , lo que implica que la precisión para un solo paso de tamaño δu será $\delta u\eta$. Así entonces,

$$\frac{1}{30}|f_1 - f_2| \left(\frac{\delta u}{\delta t} \right)^5 = (\delta u)\eta \quad (8.42)$$

En donde

$$\delta u = (\delta t) \left(\frac{30(\delta t)\eta}{|f_1 - f_2|} \right)^{(1/4)} = h\rho^{(1/4)} \quad (8.43)$$

con $\rho = 30(\delta t)\eta/|f_1 - f_2|$, que es la tasa de la precisión objetivo $(\delta t)\eta$ y la precisión actual $\frac{1}{30}|f_1 - f_2|$.

8.8. Inversión Temporal

EN FÍSICA Y ASTRONOMÍA, NUMEROSOS MÉTODOS DE INTEGRACIÓN EMPLEADOS PARA RESOLVER ECUACIONES DIFERENCIALES DEBEN CUMPLIR CON REQUISITOS ESPECÍFICOS. Uno de los aspectos importantes en la resolución de ecuaciones diferenciales asociadas a estas disciplinas es que la solución debe ser capaz de regresar.^{en} el tiempo, un concepto también conocido como inversión temporal.

La inversión temporal nos permite retroceder en nuestros pasos, reemplazando el paso de tiempo h por su valor negativo, $-h$. De esta manera, podemos regresar a nuestra condición inicial partiendo desde cualquiera de los puntos que han sido solución de la ecuación, simplemente cambiando el signo de h . La relevancia de este fenómeno radica en su relación con la conservación de la energía, un principio fundamental en física y astronomía.

Por ejemplo, en mecánica clásica, las ecuaciones de movimiento de Newton están dadas por la segunda ley de Newton:

$$m \frac{d^2 \mathbf{r}}{dt^2} = \mathbf{F}(\mathbf{r}, t) \quad (8.44)$$

donde m es la masa del objeto, $\mathbf{r}$ es la posición, t es el tiempo, y $\mathbf{F}$ es la fuerza que actúa sobre el objeto. Si invertimos el tiempo ($t \rightarrow -t$), las ecuaciones de movimiento siguen siendo válidas, lo que demuestra la simetría temporal.

En el caso de un sistema conservativo, donde la fuerza se deriva de un potencial, la energía mecánica total (energía cinética T más energía potencial V) se conserva:

$$E = T + V = \frac{1}{2}m\mathbf{v}^2 + V(\mathbf{r}) = \text{constante} \quad (8.45)$$

Aquí, la conservación de la energía está directamente relacionada con la simetría temporal. Si el sistema evoluciona hacia atrás en el tiempo, la energía total sigue siendo constante, lo que confirma la invariancia temporal de las leyes de la física.

Este concepto también se extiende a otras áreas de la física, como la mecánica cuántica y la relatividad. En general, la simetría temporal y la conservación de la energía son aspectos fundamentales de la naturaleza que nos permiten comprender y predecir el comportamiento de los sistemas físicos en el tiempo.

A continuación se presentan los principales métodos que nos permiten tener conservación de la Energía, e Inversión temporal.

8.8.1. El salto de la rana

El método del salto de la rana (*leapfrog*, en inglés) es un algoritmo de integración numérica utilizado para resolver ecuaciones diferenciales ordinarias, especialmente en problemas de mecánica clásica. Este método es un integrador simpléctico ²⁸, lo que significa que conserva muy bien las propiedades geométricas de los sistemas hamiltonianos, como la conservación de la energía y el momento. Es especialmente útil en problemas de dinámica molecular y simulaciones de n cuerpos.

Supongamos que tenemos un sistema con las siguientes ecuaciones de movimiento:

$$\frac{d\mathbf{r}}{dt} = \mathbf{v}, \quad (8.46)$$

$$\frac{d\mathbf{v}}{dt} = \mathbf{a} = \mathbf{F}(\mathbf{r}, t)/m, \quad (8.47)$$

donde $\mathbf{r}$ es la posición, $\mathbf{v}$ es la velocidad, $\mathbf{a}$ es la aceleración, $\mathbf{F}$ es la fuerza y m es la masa del objeto.

El método del salto de la rana es un método de integración de segundo orden con un paso de tiempo h . Las actualizaciones se realizan en pasos alternos para la posición y la velocidad, de la siguiente manera:

1. Actualizar la velocidad a la mitad del paso de tiempo:

$$\mathbf{v}_{n+\frac{1}{2}} = \mathbf{v}_n + \frac{1}{2}h\mathbf{a}_n, \quad (8.48)$$

donde $\mathbf{a}_n = \mathbf{F}(\mathbf{r}_n, t_n)/m$.

2. Actualizar la posición utilizando la velocidad en el punto medio:

$$\mathbf{r}_{n+1} = \mathbf{r}_n + h\mathbf{v}_{n+\frac{1}{2}}, \quad (8.49)$$

3. Calcular la aceleración en la nueva posición:

$$\mathbf{a}_{n+1} = \mathbf{F}(\mathbf{r}_{n+1}, t_{n+1})/m, \quad (8.50)$$

4. Completar el paso de tiempo para la velocidad usando la aceleración en el punto medio:

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+\frac{1}{2}} + \frac{1}{2}h\mathbf{a}_{n+1}. \quad (8.51)$$

Estos pasos se repiten iterativamente hasta alcanzar el tiempo de integración deseado. El método del salto de la rana es simple y eficiente, y debido a sus propiedades simplécticas, es muy adecuado para problemas de larga duración en los que la conservación de la energía es crítica.

²⁸Los integradores symplecticos son especialmente importantes en la mecánica celestial, donde se utilizan para simular el movimiento de cuerpos celestes, como planetas y estrellas, en sistemas gravitacionales. Estos métodos numéricos ayudan a garantizar que las simulaciones sean más precisas y se mantengan estables durante largos períodos de tiempo, lo cual es esencial para el estudio de la evolución de sistemas astronómicos y la planificación de misiones espaciales.

8.8.2. El método de Verlet y Velocity Verlet

El método de Verlet y el método Velocity Verlet son algoritmos de integración numérica utilizados para resolver ecuaciones diferenciales ordinarias, especialmente en problemas de mecánica clásica. Al igual que el método del salto de la rana, estos métodos son integradores simplécticos, lo que significa que conservan las propiedades geométricas de los sistemas hamiltonianos, como la conservación de la energía y el momento. Son especialmente útiles en problemas de dinámica molecular y simulaciones de n cuerpos.

8.8.2.1. Método de Verlet

Dadas las ecuaciones de movimiento en un sistema hamiltoniano:

$$\frac{d\mathbf{r}}{dt} = \mathbf{v}, \quad (8.52)$$

$$\frac{d\mathbf{v}}{dt} = \mathbf{a} = \mathbf{F}(\mathbf{r}, t)/m, \quad (8.53)$$

donde $\mathbf{r}$ es la posición, $\mathbf{v}$ es la velocidad, $\mathbf{a}$ es la aceleración, $\mathbf{F}$ es la fuerza y m es la masa del objeto.

El método de Verlet actualiza la posición utilizando la siguiente fórmula:

$$\mathbf{r}_{n+1} = 2\mathbf{r}_n - \mathbf{r}_{n-1} + h^2\mathbf{a}_n, \quad (8.54)$$

donde h es el paso de tiempo y $\mathbf{a}_n = \mathbf{F}(\mathbf{r}_n, t_n)/m$. El método de Verlet no calcula explícitamente las velocidades, pero se pueden obtener si es necesario a partir de las posiciones:

$$\mathbf{v}_n = \frac{\mathbf{r}_{n+1} - \mathbf{r}_{n-1}}{2h}. \quad (8.55)$$

8.8.2.2. Método Velocity Verlet

El método Velocity Verlet es una variante del método de Verlet que actualiza explícitamente las velocidades. Su algoritmo es similar al del salto de la rana, con las siguientes actualizaciones:

Actualizar la posición utilizando la velocidad y la aceleración actuales:

$$\mathbf{r}_{n+1} = \mathbf{r}_n + h\mathbf{v}_n + \frac{1}{2}h^2\mathbf{a}_n, \quad (8.56)$$

Calcular la aceleración en la nueva posición:

$$\mathbf{a}_{n+1} = \mathbf{F}(\mathbf{r}_{n+1}, t_{n+1})/m, \quad (8.57)$$

Actualizar la velocidad utilizando el promedio de las aceleraciones actual y siguiente:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \frac{1}{2}h(\mathbf{a}_n + \mathbf{a}_{n+1}). \quad (8.58)$$

Estos pasos se repiten iterativamente hasta alcanzar el tiempo de integración deseado. El método Velocity Verlet es similar al método del salto de la rana en términos de precisión y conservación de propiedades simplécticas, pero con la ventaja de calcular explícitamente las velocidades en cada paso. Además, este método es fácil de implementar y se adapta bien a fuerzas que dependen del tiempo y algoritmos de control de paso adaptativo.

Ambos métodos, Verlet y Velocity Verlet, son adecuados para problemas de larga duración en los que la conservación de la energía es crítica. Su elección depende de factores como la necesidad de obtener velocidades explícitas, la eficiencia computacional y la facilidad de implementación.

En resumen, el método de Verlet y el método Velocity Verlet son algoritmos de integración numérica ampliamente utilizados en problemas de mecánica clásica que involucran

sistemas hamiltonianos. Estos métodos conservan las propiedades simplécticas y son especialmente útiles en problemas como dinámica molecular y simulaciones de n cuerpos, donde la conservación de la energía y el momento es esencial para garantizar resultados precisos y estables a lo largo del tiempo.

8.8.3. Método Bulirsch-Stoer

El método de Bulirsch-Stoer es un algoritmo de integración numérica utilizado para resolver ecuaciones diferenciales ordinarias con alta precisión. Este método es especialmente útil en problemas que requieren una precisión muy alta o en los que el paso de tiempo debe adaptarse a lo largo de la simulación. Aunque el método de Bulirsch-Stoer no es un integrador simpléctico, es adecuado para una amplia variedad de problemas, incluidos aquellos en mecánica clásica y astrofísica.

El método de Bulirsch-Stoer se basa en dos conceptos principales: la extrapolación de Richardson y la secuencia de fracciones continuas de Bulirsch. La idea básica del método es dividir el intervalo de tiempo total en subintervalos más pequeños y resolver la ecuación diferencial en estos subintervalos utilizando un método de integración de baja precisión, como el método del trapecio modificado.

Supongamos que tenemos un sistema con las siguientes ecuaciones de movimiento:

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(\mathbf{y}, t), \quad (8.59)$$

donde $\mathbf{y}$ es el vector de estado del sistema (posición, velocidad, etc.) y $\mathbf{f}$ es la función que describe el sistema.

Para implementar el método de Bulirsch-Stoer, se siguen estos pasos:

1. Dividir el intervalo de tiempo total $[t_0, t_1]$ en n subintervalos de tamaño igual, donde n es un número entero.
2. Aplicar un método de integración de baja precisión (como el método del trapecio modificado) a cada subintervalo. Esto dará una aproximación de baja precisión para el estado del sistema al final del intervalo de tiempo total.
3. Incrementar el número de subintervalos y repetir el paso 2. Hacer esto varias veces para obtener una secuencia de aproximaciones de baja precisión con diferentes números de subintervalos.
4. Utilizar la extrapolación de Richardson para combinar las aproximaciones de baja precisión y obtener una aproximación de alta precisión para el estado del sistema al final del intervalo de tiempo total.
5. Calcular el error de la aproximación y compararlo con un umbral predefinido. Si el error es demasiado grande, repetir los pasos 1-4 con un número mayor de subintervalos.
6. Utilizar la secuencia de fracciones continuas de Bulirsch para mejorar aún más la precisión y estimar el error.
7. Si el error es aceptable, avanzar al siguiente intervalo de tiempo y repetir los pasos 1-6.

El método de Bulirsch-Stoer es altamente adaptable y puede proporcionar resultados de alta precisión en problemas con soluciones que varían rápidamente en el tiempo o que requieren un control de paso adaptativo. Sin embargo, este método puede ser más lento que otros métodos de integración, como Verlet o Velocity Verlet, en problemas que no requieren una precisión extremadamente alta o en los que la conservación de las propiedades simplécticas es más importante.



Figura 8.9: Roland Zdenk Bulirsch (10 de noviembre de 1932 - 21 de septiembre de 2022) fue un matemático alemán especializado en análisis numérico. Estudió e impartió clases en la Universidad Técnica de Múnich y enseñó internacionalmente como profesor visitante. Fue coautor del libro de referencia *Introducción al Análisis Numérico* y presidente de la edición de las obras de Johannes Kepler.



Figura 8.10: Josef Stoer (nacido el 21 de junio de 1934) es un matemático alemán especializado en análisis numérico y profesor emérito del Institut für Mathematik de la Universität Würzburg. Nació en Meschede y obtuvo su doctorado en 1961 en la Johannes Gutenberg-Universität Mainz bajo la supervisión de Friedrich Ludwig Bauer y Klaus Samelson. Ha asesorado a más de 20 estudiantes de doctorado.

8.9. Un caso práctico con métodos de inversión temporal

EL OSCILADOR ARMÓNICO ES UN CONCEPTO FUNDAMENTAL EN LA FÍSICA Y LA MATEMÁTICA, YA QUE REPRESENTA UN SISTEMA QUE EXPERIMENTA UNA FUERZA RESTAURADORA PROPORCIONAL A SU DESPLAZAMIENTO RESPECTO A UNA POSICIÓN DE EQUILIBRIO. El oscilador armónico es un concepto fundamental en la física y la matemática, ya que representa un sistema que experimenta una fuerza restauradora proporcional a su desplazamiento respecto a una posición de equilibrio. Este modelo simplificado es aplicable en una amplia variedad de situaciones, desde el movimiento de una masa unida a un resorte hasta las vibraciones moleculares y las oscilaciones en circuitos eléctricos. La ecuación del oscilador armónico se describe mediante la siguiente ecuación diferencial de segundo orden:

$$\frac{d^2x}{dt^2} = -kx \quad (8.60)$$

donde x representa la posición del oscilador, t representa el tiempo y k es una constante. Además, el oscilador armónico sirve como base para comprender fenómenos más complejos y sistemas no lineales. A continuación, se presentarán soluciones numéricas para el oscilador armónico simple utilizando diferentes métodos de aproximación, como el método Leapfrog, el método de Velocity Verlet y el método de Bulirsch-Stoer. Estas aproximaciones ofrecen distintos grados de precisión y estabilidad, lo que permite analizar y comparar su desempeño en la resolución de la ecuación diferencial de segundo orden que describe el oscilador armónico.

8.9.1. El Salto de la rana

A continuación presentamos el código asociado al método del salto de la rana para el oscilador armónico.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def leapfrog(t, x0, v0, k, dt):
5      x = np.zeros(len(t))
6      v = np.zeros(len(t))
7      x[0] = x0
8      v[0] = v0
9
10     for i in range(1, len(t)):
11         x[i] = x[i - 1] + 0.5 * dt * (v[i - 1] + v0)
12         a_new = -k * x[i]
13         v[i] = v[i - 1] + 0.5 * dt * (a_new - k * x[i - 1])
14         v0 = a_new
15
16     return x, v
17
18 t = np.arange(0,10, 0.01)
19 x0 = 1
20 v0 = 0
21 k = 1
22 dt = 0.01
23
24 x, v = leapfrog(t, x0, v0, k, dt)
25
26 plt.plot(t, x, label='Leapfrog')
27 plt.xlabel('Time')
28 plt.ylabel('Position')
29 plt.legend()
30 plt.show()
31

```

8.9.2. El método de Velocity Verlet

El siguiente código corresponde a la solución del oscilador armónico utilizando el método de velocity verlet.

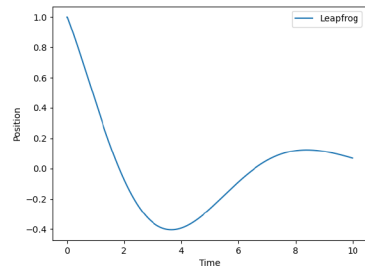


Figura 8.11: Resultado del método del salto de la rana para la ecuación diferencial de segundo orden del oscilador armónico. 8.60

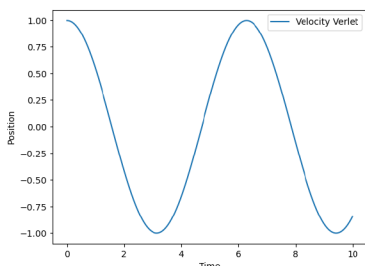


Figura 8.12: Resultado del método del

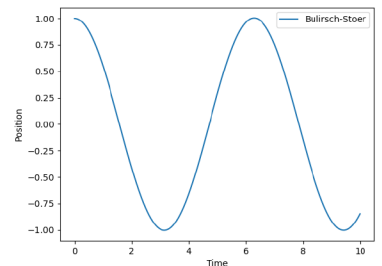
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def velocity_verlet(t, x0, v0, k, dt):
5     x = np.zeros(len(t))
6     v = np.zeros(len(t))
7     x[0] = x0
8     v[0] = v0
9
10    for i in range(1, len(t)):
11        x[i] = x[i - 1] + v[i - 1] * dt + 0.5 * (-k * x[i - 1]) * dt**2
12        a_new = -k * x[i]
13        v[i] = v[i - 1] + 0.5 * (a_new - k * x[i - 1]) * dt
14
15    return x, v
16
17 t = np.arange(0, 10, 0.01)
18 x0 = 1
19 v0 = 0
20 k = 1
21 dt = 0.01
22
23 x, v = velocity_verlet(t, x0, v0, k, dt)
24
25 plt.plot(t, x, label='Velocity Verlet')
26 plt.xlabel('Time')
27 plt.ylabel('Position')
28 plt.legend()
29 plt.show()

```

8.9.3. El método de Bulirsch-Stoer

El método de Bulirsch-Stoer es más complejo y requiere múltiples pasos tales como la extrapolación polinomial, extrapolación de Richardson, y paso de control adaptativo. Debido a esto utilizaremos una librería predefinida para una rápida y más simple implementación.



```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4
5 def harmonic_oscillator(t, y, k):
6     dydt = [y[1], -k * y[0]]
7     return dydt
8
9 def bulirsch_stoer(t, x0, v0, k):
10    sol = solve_ivp(harmonic_oscillator, [t[0], t[-1]], [x0, v0], args=(k,),
11        ↪ method='BDF', t_eval=t)
12    return sol.y[0], sol.y[1]
13
14 t = np.arange(0, 10, 0.01)
15 x0 = 1
16 v0 = 0
17 k = 1
18
19 x, v = bulirsch_stoer(t, x0, v0, k)
20
21 plt.plot(t, x, label='Bulirsch-Stoer')
22 plt.xlabel('Time')
23 plt.ylabel('Position')
24 plt.legend()
25 plt.show()
26

```

Figura 8.13: Resultado del método Bulirsch-Stoer para la ecuación diferencial de segundo orden del oscilador armónico. 8.60

Ejercicio:

Escriba un código para Bulirsch-Stoer, para el caso anterior pero no utilizando la librería previa, sino que incorporando la extrapolación polinomial, de Richardson, y paso de control adaptativo.

8.9.4. Comparación de los métodos

El siguiente código presenta una solución de la ecuación diferencial utilizando los tres métodos descritos previamente, y que cumplen con el proceso de inversión temporal.

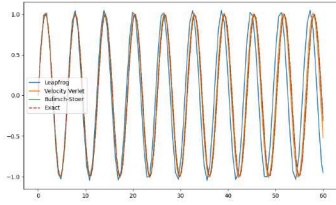


Figura 8.14: Resultado de la comparación de los resultados para la ecuación diferencial de segundo orden del oscilador armónico. 8.60

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4 from scipy.integrate import solve_ivp
5
6 # define the differential equation
7 def harmonic_oscillator(t, y):
8     return [y[1], -y[0]]
9
10 # initial conditions
11 y0 = [0, 1] # starting at rest at x=0
12
13 # time grid
14 t = np.linspace(0, 60, 200)
15
16 # Leapfrog method
17 def leapfrog(t, y0):
18     h = t[1] - t[0] # time step
19     n = len(t)
20     y = np.zeros((n, 2))
21     y[0] = y0
22
23     # first step: Euler method
24     y[1] = y[0] + h * np.array(harmonic_oscillator(t[0], y[0]))
25
26     for i in range(1, n-1):
27         y[i+1, 0] = y[i-1, 0] + 2*h*y[i, 1]
28         y[i+1, 1] = y[i-1, 1] - 2*h*y[i, 0]
29
30     return y[:, 0]
31
32 # Velocity Verlet method
33 def velocity_verlet(t, y0):
34     h = t[1] - t[0] # time step
35     n = len(t)
36     y = np.zeros((n, 2))
37     y[0] = y0
38
39     for i in range(n-1):
40         y[i+1, 0] = y[i, 0] + h*y[i, 1] + 0.5*h**2*(-y[i, 0])
41         y[i+1, 1] = y[i, 1] + 0.5*h*(-y[i, 0] - y[i+1, 0])
42
43     return y[:, 0]
44
45 # Bulirsch-Stoer method
46 def bulirsch_stoer(t, y0):
47     sol = solve_ivp(harmonic_oscillator, [t[0], t[-1]], y0, t_eval=t, method='BDF')
48     return sol.y[0]
49
50 # compute solutions
51 y_leapfrog = leapfrog(t, y0)
52 y_verlet = velocity_verlet(t, y0)
53 y_bulirsch_stoer = bulirsch_stoer(t, y0)
54
55 # exact solution
56 y_exact = np.sin(t)
57
58 # plot the results

```

```
59 plt.figure(figsize=(10,6))
60 plt.plot(t, y_leapfrog, label='Leapfrog')
61 plt.plot(t, y_verlet, label='Velocity Verlet')
62 plt.plot(t, y_bulirsch_stoer, label='Bulirsch-Stoer')
63 plt.plot(t, y_exact, label='Exact', linestyle='dashed')
64 plt.legend()
65 plt.show()
66
```

8.10. Perspectivas y bibliografía recomendada

Este capítulo ha presentado una introducción a los métodos numéricos más utilizados para resolver ecuaciones diferenciales ordinarias: el método de Euler, los métodos de Runge–Kutta de distintos órdenes y los integradores específicos para ecuaciones de segundo orden como Leapfrog, Velocity Verlet y Bulirsch–Stoer. Cada uno de estos métodos supone un compromiso entre facilidad de implementación, coste computacional y precisión. Es importante comprender sus hipótesis de estabilidad, sus órdenes de error y las situaciones en las que resultan adecuados.

El libro *Numerical Methods for Ordinary Differential Equations* de John C. Butcher ofrece una revisión moderna y exhaustiva de estos métodos. La segunda edición amplía el tratamiento de los métodos de Runge–Kutta y los métodos multietapa e incorpora una gran variedad de ejercicios y ejemplos. Butcher subraya que los integradores de orden alto permiten obtener una precisión muy elevada con pasos de tiempo relativamente grandes, siempre que se mantenga el control de errores y la estabilidad de los algoritmos.

Una introducción concisa al método de Runge–Kutta de cuarto orden puede encontrarse en el material didáctico de Swarthmore College. Allí se explica que RK4 evalúa cuatro pendientes en puntos intermedios dentro del intervalo y que combina estas pendientes mediante un promedio ponderado para obtener una aproximación más precisa al valor de la solución [14]. Esta referencia destaca por qué RK4 se ha convertido en uno de los métodos más utilizados en la práctica.

Por otro lado, los métodos numéricos constituyen la puerta de entrada al estudio de sistemas no lineales y caóticos. El texto de Steven Strogatz, *Nonlinear Dynamics and Chaos*, destaca cómo pequeñas variaciones en parámetros o condiciones iniciales pueden dar lugar a comportamientos cualitativamente diferentes, y muestra aplicaciones en física, biología y electrónica. Una comprensión sólida de los integradores numéricos permite explorar estos fenómenos sin que la elección del método introduzca artefactos.

Para quienes deseen profundizar, se recomienda también revisar artículos especializados y documentación de bibliotecas como SciPy, que implementan métodos avanzados con control adaptativo del paso de tiempo. En la práctica profesional, la elección del integrador se basa en el equilibrio entre eficiencia y precisión, así como en la naturaleza de la ecuación a resolver.